St. Anne's Arts and Science College

Department of Computer Science

Team Members	Register No	Project Name	Phase Topic
JAISREE.R	222409465	FitTrack AI - Personalized Fitness Recommendations Powered by AI	Phase 3 PROJECT DESIGN
JANU SRI.D	222409466		
JECHINTHA JOYCE.S	222409467		
JENITA.G	222409468		

FITTRACK AI

Personalized Fitness Recommendations Powered by AI

PHASE 3

PROJECT DESIGN PHASE

Project Name: AI FitTrack

Technology Stack: Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt.js, Google Gemini AI

CONTENTS

Section	Topic
3.1	Design Overview
3.2	MVC Architecture
3.3	System Components
3.3.1	Client Layer
3.3.2	Express Server
3.3.3	API Routes
3.3.4	Authentication Middleware
3.3.5	Controller Layer
3.3.6	Service Layer
3.3.7	Database Layer
3.4	Database Design
3.5	User–Workout Relationship
3.6	User Entity Design
3.7	Workout Entity Design
3.8	API Design
3.9	AI Service Design
3.10	Security Design
3.11	Phase Summary

PHASE 3 – PROJECT DESIGN PHASE

3.1 Design Overview

The design phase defines the structure of AI FitTrack and explains how its components cooperate to provide secure workout management and AI-assisted fitness services. The backend is organized using a layered Model–View–Controller (MVC) architecture. This separation assigns authentication, workout management, database interaction, and AI integration to appropriate modules.

The application follows a request–response workflow. A client sends an HTTP request to the Express server, where the request is routed through the relevant API route and, when required, authentication middleware. The controller coordinates the requested operation with the service or data layer. The system then returns the result to the client in JSON format.

3.2 MVC Architecture

AI FitTrack follows the Model–View–Controller (MVC) architectural approach to separate data management, API interaction, and application logic.

3.2.1 Model

The Model layer defines the structure of application data and supports database interaction. Mongoose models are used to define and manage the User and Workout data stored in MongoDB.

3.2.2 View

As AI FitTrack is primarily a backend REST API, the View is represented by the API interface and the applications or tools that consume it. Postman and Thunder Client can be used to submit requests and inspect the JSON responses returned by the backend.

3.2.3 Controller

The Controller layer handles operations requested through the API. It receives requests from the route layer, performs the required validation and processing, interacts with the relevant services or models, and returns an appropriate response to the client.

3.3 System Components

The backend is divided into components with clearly defined responsibilities. This modular structure supports separation of concerns across request handling, authentication, business operations, data storage, and AI services.

3.3.1 Client Layer

Sends requests to backend APIs. The client may be a web application, mobile application, or API testing tool such as Postman. For protected operations, it includes the authentication token with the request.

3.3.2 Express Server

Acts as the main backend gateway. It initializes the application, processes JSON requests, enables CORS, registers API routes, and handles middleware and errors.

3.3.3 API Routes

Organizes the available API endpoints into major groups for authentication, workout management, and AI-based services.

3.3.4 Authentication Middleware

Processes protected requests by reading the JWT from the Authorization header, verifying the token, identifying the associated user, and allowing the request to continue when authentication is valid.

3.3.5 Controller Layer

Coordinates each API operation by connecting the incoming request with the appropriate database or service operation and returning the result to the client.

3.3.6 Service Layer

Contains supporting application services, including JWT handling, password-related operations, and communication with the Google Gemini AI service.

3.3.7 Database Layer

Uses MongoDB as the primary database and Mongoose as the Object Data Modeling (ODM) layer for defining schemas and interacting with stored data.

3.4 Database Design

The database design is centered on two main collections: Users and Workouts. The Users collection stores account information required for user management, while the Workouts collection stores individual workout records and their association with a user.

- Users collection: stores the user's name, email address, encrypted password, and creation and update timestamps.
- Workouts collection: stores workout details such as workout name, category, duration, calories burned, workout date, and the user associated with the record.

3.5 User–Workout Relationship

A one-to-many relationship is maintained between the User and Workout entities. One user can have multiple workout records, while each workout record belongs to one authenticated user. The relationship is represented by storing the user's ObjectId as a reference in the Workout collection.

3.6 User Entity Design

The User entity contains the fields required for account management and authentication.

Field	Purpose
_id	Unique identifier for the user.
name	Stores the user's name.
email	Stores the user's unique email address.
password	Stores the encrypted password.
createdAt	Records the time when the user record was created.
updatedAt	Records the time when the user record was most recently updated.

The email field is maintained as unique. Before a password is stored, it is protected using bcrypt.js.

3.7 Workout Entity Design

The Workout entity stores the information required to maintain individual workout records.

Field	Purpose
_id	Unique identifier for the workout record.
user	Reference to the user associated with the workout.
workoutName	Name of the workout.
category	Type or category of workout.
duration	Duration of the workout.
caloriesBurned	Calories recorded for the workout.
workoutDate	Date on which the workout took place.
createdAt	Record creation time.
updatedAt	Time of the latest record update.

The supported workout categories include Cardio, Strength Training, Yoga, Running, Cycling, and Walking. Validation rules are applied to fields such as duration and calories burned.

3.8 API Design

AI FitTrack follows a RESTful API structure. The endpoints are organized into three major groups according to their purpose:

- Authentication APIs: support user registration, login, and profile access.
- Workout APIs: support adding, viewing, searching, updating, and deleting workout records.
- AI APIs: provide personalized workout recommendations and AI-generated fitness insights.

Protected endpoints require valid authentication before the requested operation is processed.

3.9 AI Service Design

The AI service is designed as a separate backend component. Google Gemini is used to generate personalized fitness recommendations based on information supplied by the user.

The recommendation process uses details such as age, fitness goal, and experience level. Fitness insights are generated from workout statistics, including total workouts, average workout duration, and total calories burned.

3.10 Security Design

Security is incorporated into the system design through password protection, JWT-based authentication, and protected API routes.

- Passwords are encrypted using bcrypt.js before they are stored in the database.
- JWT is used to authenticate users who access protected resources.
- Authentication middleware verifies the JWT received through the Authorization header before allowing access to protected operations.

- Workout-related operations are associated with the authenticated user, supporting user-specific access to workout information.